Software Engineering Assignment

Q1. Case Study: SmartClinic System

A small clinic wants to develop a **SmartClinic System** to manage appointments, patient records, and doctor schedules. Patients should be able to register, book appointments, and view their reports online. Doctors need access to patient history and must be able to update diagnoses and prescriptions. The receptionist will manage walk-in patients and update basic records. The system should support online payments and send notifications for appointments and report availability.

The clinic wants the system to be secure, easy to use, and quick in retrieving patient data. It must also remain available during peak hours and support both mobile and web access.

From the above case study, identify:

(a) Functional Requirements

(b) Non-Functional Requirements

Q2. Consider the following pseudo-code for a function **CheckEligibility(x, y)**, where
x = student's attendance percentage and
y = average assignment score:

```
CheckEligibility(x, y):
1.  if (x < 0 or x > 100) then
2.      print "Invalid attendance"
3.      return 0
4.  else
5.      if (x < 75) then
6.          print "Attendance shortage"
7.          if (y >= 50) then
8.              print "Assignment score compensates"
9.              return 1
10.         else
11.             print "Not eligible"
12.             return 0
13.     else
14.         if (y < 40) then
15.             print "Low assignment score"
16.             return 0
17.         else
18.             print "Eligible"
19.             return 1
```

(a) Draw the Control Flow Graph (CFG)

Create nodes for each major block and show all the possible branches.

(b) Calculate the Cyclomatic Complexity (V(G))

Use any of the three methods:

- **V(G) = E – N + 2**
- **V(G) = P + 1** (where P = number of predicate nodes / decisions)
- **V(G) = Number of Regions**

(c) Identify all Independent Paths

List each distinct, linearly independent path through the program based on your Cyclomatic Complexity value.

(d) Design Test Cases

Prepare test cases that cover all the independent paths.
Each test case should specify:

- Input values (x, y)
- The path taken
- Expected output / message

Q3. Compare **Waterfall**, **Incremental**, and **Spiral** life cycle models for building a mobile-based **Fitness Tracking Application**. Justify which model is best suited and why, considering project risk, user involvement, and delivery timelines.

Q4. Project Estimation: PERT & CPM

A software project consists of the following activities with optimistic (O), most likely (M), and pessimistic (P) time estimates:

Activity	Predecessor	O	M	P
A	—	1	2	3
B	A	2	3	10
C	A	1	4	7
D	B	2	5	8
E	B, C	3	4	5
F	D, E	1	2	3

(a) Calculate the **expected time (TE)** and **variance** for each activity using PERT formulas.
(b) Draw the **PERT network diagram**.
(c) Determine the **critical path** and compute the **expected project completion time**.
(d) Compute the **probability** of completing the project within **18 days**.

Q5. CPM (Critical Path Method)

A project has the following activities and their durations:

Activity	Predecessor	Duration (days)
A	—	4
B	A	5
C	A	6
D	B	7
E	C	4
F	D, E	3

(a) Draw the **CPM network diagram**.
(b) Calculate the **earliest start (ES)**, **earliest finish (EF)**, **latest start (LS)**, **latest finish (LF)**, and **slack** for each activity.
(c) Identify the **critical path**.
(d) Calculate the **minimum project completion time**.

Q6. Software Design

Explain the concepts of **modularity**, **abstraction**, and **architecture styles**.
Then propose a suitable architecture (layered, client-server, microservices, etc.) for an **Online Examination System**, with justification.

Q7. Testing Strategies

Generate **five black-box test cases** using any type of Black box testing for the password validation function of a **User Login Module**, considering conditions like length, special characters, and case sensitivity.

Q8. Software Maintenance

A deployed **Campus Attendance Management System** frequently crashes during peak hours. Identify the possible types of maintenance (corrective, adaptive, perfective, preventive) required and justify each with examples from the scenario.

Q9. Explain Capability Maturity Model with diagram.

Q10. CASE STUDY: SmartCampus Connect System

Manipal University Jaipur wants to develop a **SmartCampus Connect System (SCCS)**—a unified application that integrates **attendance, timetable, course materials, announcements, fee payment, event registration, and complaint management** for students and faculty.

The university administration has requested a system that works on both **web** and **mobile** platforms. Students, faculty, and administrators will have different levels of access.

Functional Requirements

1. User Management

- Students and faculty can register/login using university credentials.
- Forgot-password and two-factor authentication must be supported.
- Admin can add/edit/disable users.

2. Attendance Management

- Faculty can mark attendance for each class session.
- Students can view subject-wise attendance reports.
- The system sends automatic alerts when attendance falls below a threshold.

3. Timetable & Academic Resources

- Students and faculty can view their personalized timetable.
- Faculty can upload lecture notes, assignments, and recordings.
- Students can download/view these resources.

4. Notifications & Announcements

- Admin and faculty can broadcast alerts (exam dates, events, deadlines).
- Students receive push notifications on mobile.

5. Fee Payment Module

- Students can view pending fees.
- System integrates with an online payment gateway.
- Digital receipts generated automatically.

6. Events & Club Activities

- Students can browse upcoming events.
- Register for workshops/competitions.
- Organizers can approve/reject registrations.

7. Complaint & Support System

- Students can raise complaints (IT issues, hostel, academics).
- Admin assigns these to relevant departments.
- Students track status (Pending → In-Process → Resolved).

Create following UML diagrams:

1. **Use Case Diagram**
   Identify actors and major use cases for SmartCampus Connect.
2. **Class Diagram**
   Show classes such as User, Student, Faculty, Attendance, Timetable, CourseMaterial, Event, Complaint, Payment, Notification, etc.
3. **Object Diagram**
   Create object instances for a sample scenario (e.g., a student registering for an event while downloading notes).

4. **Sequence Diagram**
   Choose one scenario (e.g., fee payment, attendance marking, complaint resolution) and show message flow.

5. **Activity Diagram**
   Model any process such as "Student Event Registration", "Attendance Marking", or "Complaint Handling".

6. **State Machine Diagram**
   Show states for one entity—for example: **Complaint** (Submitted → Assigned → In-Process → Resolved → Closed).

7. **Communication Diagram**
   Represent interactions between objects for any selected use case.

8. **Component Diagram**
   Show major components such as Authentication Service, Attendance Module, Payment Module, Notification Module, DB Server, API Gateway, etc.

9. **Deployment Diagram**
   Show nodes: Mobile Device, Web Browser, Application Server, Database Server, Payment Gateway Server, Notification Server.